

PostIT Project Problem Statement

PostIT – Project Problem Statement

Overview

Build **PostIT**, a full-stack social media web application using the **MERN stack** (MongoDB, Express.js, React, Node.js). The app allows registered users to log in, share short posts, like or unlike posts made by other users, and update their profile. The project covers the complete flow from database design to REST API development to React UI with global state management using Redux Toolkit.

Technology Stack

Layer	Technology
Frontend	React, Redux Toolkit, React Router DOM, Reactstrap (Bootstrap)
Backend	Node.js, Express.js
Database	MongoDB Atlas (cloud) via Mongoose
Form Validation	React Hook Form + Yup
Password Security	bcrypt
HTTP Client	Axios

MongoDB Collections & Data Models

Collection: `userInfos` — `UserModel`

Stores one document per registered user.

Field	Type	Notes
name	String	Required
email	String	Required, unique
password	String	Required, stored as bcrypt hash

Field	Type	Notes
profilePic	String	Optional, URL to profile image

Collection: posts — PostModel

Stores one document per post. Uses Mongoose timestamps to auto-record createdAt.

Field	Type	Notes
postMsg	String	Required, the post content
email	String	Required, email of the author
likes.count	Number	Default 0, total like count
likes.users	[String]	Array of user IDs who liked the post
createdAt	Date	Auto-generated by Mongoose timestamps

REST APIs — server/index.js

The Express server runs on **port 3001** and connects to MongoDB Atlas on startup.

Method	Endpoint	Purpose	Algorithm
POST	/registerUser	Register a new user	Receive name/email/password → hash password with bcrypt (10 salt rounds) → save new UserModel document → return saved user
POST	/login	Authenticate a user	Receive email/password → find user by email → compare password to hash using bcrypt → return user object on success, error on failure
POST	/logout	Log out a user	Return a success message (session/token management can be extended here)
POST	/savePost	Create a new post	Receive postMsg/email → save new PostModel document → return saved post
GET	/getPosts	Fetch all posts	Find all posts sorted by createdAt descending (newest first) → return posts array and total count
PUT	/likePost/:postId	Toggle like/unlike on a post	Find post by ID → check if userId is in likes.users array → if yes: decrement count and remove userId

Method	Endpoint	Purpose	Algorithm
PUT	/updateUserProfile/:email	Update user profile	(\$inc, \$pull) → if no: increment count and add userId (\$inc, \$addToSet) → return updated post Find user by email → update name and profilePic → if password changed, hash the new one with bcrypt → save and return updated user

Redux Store — Store/store.js

Configure the Redux store using configureStore from Redux Toolkit. Register two reducers:

- users → from UserSlice
- posts → from PostSlice

Wrap the entire React app in a `<Provider store={store}>` in `index.js` so all components can access the store.

Redux Slice 1: UserSlice — Features/UserSlice.js

State shape:

```
{ user: {}, isLoading: false, isSuccess: false, isError: false }
```

Thunks (Async Actions)

Thunk	API Call	What it does
registerUser(userData)	POST /registerUser	Sends name/email/password to server, returns new user
login(userData)	POST /login	Sends email/password, returns authenticated user object
logout()	POST /logout	Calls logout endpoint
updateUserProfile(userData)	PUT /updateUserProfile/:email	Sends updated name, password, profilePic

Extra Reducers (State Changes per Thunk Lifecycle)

Each thunk has three lifecycle cases — pending, fulfilled, rejected:

- **pending** → set `isLoading: true`, reset `isError` and `isSuccess`
- **fulfilled** → store returned data in `state.user`, set `isLoading: false`, `isSuccess: true`
- **rejected** → set `isLoading: false`, `isError: true`
- On **logout fulfilled** → clear `state.user` back to `{}`

Redux Slice 2: PostSlice — Features/PostSlice.js

State shape:

```
{ posts: [], comments: [], likes: [] }
```

Thunks

Thunk	API Call	What it does
<code>savePost(postData)</code>	POST <code>/savePost</code>	Sends <code>postMsg/email</code> , returns the new post
<code>getPosts()</code>	GET <code>/getPosts</code>	Returns all posts sorted newest first
<code>likePost(postData)</code>	PUT <code>/likePost/:postId</code>	Sends <code>postId/userId</code> , returns updated post

Extra Reducers

- **savePost fulfilled** → prepend the new post to the front of `state.posts` using `unshift`
- **getPosts fulfilled** → replace `state.posts` with the full array from the server
- **likePost fulfilled** → find the post in `state.posts` by `_id` and update its `likes` property with the returned value

React Components

App.js — Root Component & Router

Purpose: Defines all client-side routes and conditionally renders the Header.

Algorithm: Use `useSelector` to check if a user email exists in Redux state. If logged in, show `<Header>`. Define routes: `/` → Home, `/login` → Login, `/register` → Register, `/profile` → Profile.

index.js — App Entry Point

Purpose: Mounts the React app into the DOM.

Algorithm: Wrap `<App>` in `<Provider store={store}>` so all components can access the Redux store via `useSelector` and `useDispatch`.

Header.js — Navigation Bar

Purpose: Shows navigation links and the logged-in user's name and email.

Algorithm: Use `useSelector` to get the user from Redux. Provide links to Home and Profile. On Logout click, dispatch the logout thunk and redirect to `/login` using `useNavigate`.

Footer.js — Page Footer

Purpose: Display a simple copyright footer at the bottom of every page.

Home.js — Home Page (Protected)

Purpose: The main page shown after login — displays the user info panel, post creation area, and the posts feed.

Algorithm: Use `useSelector` to get email. Use `useEffect` to redirect to `/login` if no user is logged in. Render `<User>`, `<SharePosts>`, and `<Posts>` in a grid layout.

Login.js — Login Form

Purpose: Allow a registered user to log in.

Algorithm:

1. Manage email and password in local state with `useState`.
 2. On button click, dispatch the `login(userData)` thunk.
 3. Use `useEffect` watching `isSuccess` and `isError` from Redux state.
 4. On success → navigate to `/ (home)`. On error → stay on `/login` and show error message.
-

Register.js — Registration Form

Purpose: Allow a new user to create an account with validated inputs.

Algorithm:

1. Use `react-hook-form` with `yupResolver` connecting the `userSchemaValidation` Yup schema.
2. Yup rules: name required, valid email format, password 4–20 chars, `confirmPassword` must match password.
3. On valid submit, dispatch `registerUser(userData)` thunk and redirect to `/login`.

Profile.js — Update Profile (Protected)

Purpose: Let the logged-in user update their name, password, and profile picture URL.

Algorithm:

1. Pre-fill form fields from the Redux user object using `useState` initial values.
 2. On submit, validate that password and `confirmPassword` match.
 3. Dispatch `updateUserProfile(userData)` thunk with email (read-only), name, password, and `profilePic`.
 4. Redirect back to `/profile` after update.
-

User.js — User Info Panel

Purpose: Display the logged-in user's profile picture, name, and email.

Algorithm: Use `useSelector` to read name, email, and `profilePic` from Redux. If `profilePic` is set, show it; otherwise show the default avatar image.

SharePost.js — Create a Post

Purpose: Provide a textarea for the user to type and submit a new post.

Algorithm:

1. Manage `postMsg` in local state.
 2. On "PostIT" button click, validate that the message is not empty.
 3. Dispatch `savePost({ postMsg, email })` thunk.
 4. Clear the textarea after posting.
-

Posts.js — Posts Feed

Purpose: Display all posts with timestamps, like counts, and a like/unlike toggle button.

Algorithm:

1. On component mount (`useEffect`), dispatch `getPosts()` to load all posts from the server.
 2. Render each post in a table row: show author email, relative time using `moment.js` (`fromNow()`), post message, and a thumbs-up icon with the like count.
 3. On like icon click, dispatch `likePost({ postId, userId })` thunk. The server toggles like/unlike automatically based on whether the user has already liked the post.
-

UserValidations.js — Form Validation Schema

Purpose: Define validation rules shared across Register (and optionally other) forms using Yup.

Field	Rules
name	Required string
email	Required, valid email format
password	Required, minimum 4 characters, maximum 20 characters
confirmPassword	Required, must match the password field exactly

Application Flow Summary

User opens app

└ Not logged in → redirect to /login

└ Login success → navigate to / (Home)

└ SharePost: type message → PostIT → POST /savePost → new post

added to feed

└ Posts: load all posts → GET /getPosts → display feed

└ click like → PUT /likePost/:id → toggle like in DB +

update Redux

└ Header → Profile link → /profile → update name/password/pic →

PUT /updateUserProfile

└ Header → Logout → POST /logout → clear Redux user → redirect to /login

└ No account → /register → fill form → validation → POST /registerUser → redirect to /login